Oblig 2: The `AeroScript` Language

Deadline: 9 October 2025

1 Objective

For your second assignment, you will extend the abstract syntax tree (AST) that forms the structure of `AeroScript`. You will take on the challenge of implementing both a lexer and a parser to analyze and execute `AeroScript` code. Following this, you will develop a complete interpreter by expanding on the expression evaluator you made in Assignment 1.

2 Syntax

For easy reference, you can find the concrete syntax of `AeroScript` in Figure 1.

$$
\begin{array}{rcl}
 & & n \in \mathbb{R}^+ \\
 & & id \in \text{NAME} \\
\textit{program} & := & (\textit{mode})^+ \\
\textit{mode} & := & \text{`}\rightarrow\text{`}^? \ \textit{id} \ \text{`}\{\text{`} \ (\textit{statement})^* \ \text{`}\}\text{`} \ (\text{`}\rightarrow\text{`} \ \textit{id})^? \\
\textit{statement} & := & \textit{action} \\
\textit{action} & := & (\textit{acDock} \ | \ \textit{acMove} \ | \ \textit{acTurn} \ | \ \textit{acAscend} \ | \ \textit{acDescend}) \\
 & & \ (\text{`}\texttt{for}\text{`} \ \textit{expression} \ \text{`}\texttt{seconds}\text{`} \ | \ \text{`}\texttt{at speed}\text{`} \ \textit{expression})^? \\
\textit{acDock} & := & \text{`}\texttt{return to base}\text{`} \\
\textit{acMove} & := & \text{`}\texttt{move}\text{`} \ (\text{`}\texttt{to}\text{`} \ \text{`}\texttt{point}\text{`} \ \textit{point} \ | \ \text{`}\texttt{by}\text{`} \ n) \\
\textit{acTurn} & := & \text{`}\texttt{turn}\text{`} \ (\text{`}\texttt{right}\text{`} \ | \ \text{`}\texttt{left}\text{`})^? \ \text{`}\texttt{by}\text{`} \ \textit{expression} \\
\textit{acAscend} & := & \text{`}\texttt{ascend by}\text{`} \ \textit{expression} \\
\textit{acDescend} & := & \text{`}\texttt{descend by}\text{`} \ \textit{expression} \ | \ \text{`}\texttt{descend to ground}\text{`} \\
\textit{expression} & := & \text{`}--\text{`}\textit{expression} \,|\, \textit{expression} \ \oplus \ \textit{expression} \,|\, \text{`}\texttt{random}\text{`} \ \textit{range}^? \\
 & & \ |\, \text{`}\texttt{point}\text{`} \ \textit{point} \,|\, n \,|\, \text{`}(\text{`}\textit{expression}\text{`})\text{`} \\
\textit{range} & := & \text{`}[\text{`}\textit{expression}\text{`},\text{`}\textit{expression}\text{`}]\text{`} \\
\textit{point} & := & \text{`}(\text{`}\textit{expression}\text{`},\text{`}\textit{expression}\text{`})\text{`} \\
\oplus & := & \text{`}+\text{`} \,|\, \text{`}-\text{`} \,|\, \text{`}*\text{`}
\end{array}
$$

Figure 1. The syntax of `AeroScript`. The star ($\odot^*$) denotes a sequence containing 0 or more occurrences. and the question mark ($\odot^?$) denotes choice (zero or one occurrence), the plus ($\odot^+$) denotes a sequence containing one or more occurrences. NAME is the set of all possible identifiers.

Intuition: An `AeroScript` program consists of one or more *modes*, which have some similarity to methods in Java. Each mode is defined by an identifier and enclosed actions within braces. A mode may be prefixed by an optional arrow and may end with an optional arrow and a named mode. If a mode is prefixed by an arrow, this mode should be automatically started as part of the program's run block. If a mode ends with an arrow to another mode, program execution should transition to this mode and execute thew corresponding instructions, before continuing with the rest of the main block. The following are some example modes:

```
-> MissionControl {} -> TakeOffLand

TakeOffLand {
  ascend by 20
  move to point (50, 50)
```

```
} -> ScanArea

ScanArea {
  move to point (100, 100) at speed 10
} -> EndMission

EndMission { return to base }


-> RandomTour {} -> TakeOff

TakeOff {
  ascend by 20
  move to point (random[0, 100], random[0, 100])
} -> Explore

Explore {
  move to point (random[0, 100], random[0, 100]) at speed 10
}  -> ReturnToBase

ReturnToBase { return to base }
```

When executing an `AeroScript` program, your program should read the content of the file `program.aero` (can be found under `src/test/resources/`) and parse it into an AST. Your program should then execute the `AeroScript` program by traversing the AST and executing the statements.

Note: the modes that are defined (e.g. `TakeOffLand`, `TakeOff`, etc.) **must not** be executed when not invoked (e.g. `TakeOffLand` gets executed when we receive the message `launch`).

3 Semantics of `AeroScript`

The syntax and semantics of `AeroScript` expressions were covered (and hopefully implemented) in your previous assignment; we now focus on how to evaluate modes.

Every mode is given by a sequence of actions. When a mode is entered, its actions are executed in order. Thus, the program transitions to the next statement in the sequence. At all times the program needs to keep track at the current

- position,
- altitude,
- distance traveled, and
- battery level.

If the level of the battery drops to zero, the program should trigger an exception.

Each action needs to be printed out to the command line as a log, and will affect the variables of the program (be it the position as a coordinate in X and Y or the altitude if ascending or descending) and will deplete the battery level.

3.1 Detailed semantics

Modes and Transitions In `AeroScript`, **modes** are like different states or scenarios in which a drone can operate—such as taking off, flying, or landing and are defined in the grammar by the `mode`. In Assignment 2, a mode is similar to a method and a new method can only be called after executing the method body of the active method.

Each mode contains a set of operations that can be executed when the drone is in that state. For instance, a `TakeOff` mode might include instructions to power up the rotors and ascend to a specific altitude. Note that in order to have a program that makes the drone *do* something, it must have at least one mode with a leading transition arrow—the *start mode*.

Actions and Commands Actions are the building blocks of what your drone can actually do. These include moving to a specific (2D) coordinate, ascending or descending, and turning. Each action in `AeroScript` directly corresponds to a command sent to the drone's control system.

Action	Description	Battery cost
acDock	Terminal action that takes no arguments. The program should print the distance travelled and the battery level before it terminates.	altitude + (time * 0.1) + (speed * 1)
acMove	Move to a point or by *n* meters. If point, we need to compute the cartesian difference (in 2 dimension of point, do not consider the altitude), otherwise we move by *n* meters into either the *x* or *y* directions (choose one).	(distance * 0.7 if point, distance * 0.5 otherwise) + (time * 0.1) + (speed * 1)
acTurn	Turn by *n* degrees.	angle * 0.3 + (time * 0.1) + (speed * 1)
acAscend	Go up by *n* meters.	distance * 0.6 + (time * 0.1) + (speed * 1)
acDescend	Go to the ground (`altitude` goes to 0).	`altitude` * 0.2 + (time * 0.1) + (speed * 1)

Table 1. Table of costs for battery depletion. Note that for time and speed they might be not present at all — set the value to 0 if the parameter is not present.

- Commands like `move to point (x, y)` tell the drone to fly to a specific location in the two-dimensional plane at its current altitude. The semantics of this command involve calculating the flight path and adjusting the rotors to navigate to these coordinates.

Expressions and Calculations Expressions and calculations was covered in the last assignment (Oblig 1); no changes are warranted here!

Battery Depletion Each action (except printing to the command line) will deplete the battery level as given in Table 1.

Distance travelled, Each action should update the distance travelled as described in Table 1.

3.2 Putting It All Together

After parsing the source into an AST, your interpreter should traverse it and execute the instructions. More specifically, it should identify the start mode, executes actions, and evaluate expressions. The interpreter continually checks which mode the drone is in, and which actions to execute based on the current mode.

Note: Moreover, the modes that are defined (e.g., `TakeOffLand`, `TakeOff`, etc.) **must not** be executed when not invoked (e.g. `TakeOffLand` gets executed after the body of mode `MissionControl` has been completed. Modes that start with '`->`' should be executed when reached. Modes that do not start with '`->`' should only be executed when called from another mode.

4 Your task

The assignment includes a template for the implementation, similar to Oblig 1. You should do the following:

- Extend the AST to cover the entire syntax of `AeroScript`.
- Extend the lexer and parser to cover the language given in Figure 1 using `ANTLR`.
- Extend the interpreter to cover the language given in Figure 1.
- We have provided some `JUnit` tests. **Discuss the coverage of the tests**, and provide your own. In particular, make sure that exceptions are thrown correctly.
- Write a report (2-3 pages) where you discuss the challenges you faced and choices you made during implementation. Be sure to read the report guide on the course page beforehand.

5 Implementation Hints

- When adding elements to the heap, you should use these variables
  - `altitude` (initially set to 0)
  - `battery level`
  - `distance travelled` (initially set to 0)
  - `initial mode` (initially set to null)
- When implementing the execution of modes:
  - When you start running a mode, you need to push a copy of the corresponding code from the `EXECUTION_TABLE` on the stack
  - Think about what should happen when the drone runs out of battery…
  - make sure your code can run recursive modes (until they run out of battery!)
- When implementing the events, make sure to add the correct logic for each event:
  - `obstacle` should trigger the function corresponding to the event immediately
  - `low battery` should trigger the function corresponding to the event if the battery level is below 20
  - `message` should trigger the function corresponding to the event if the message is received from the REPL

6 What to Submit

You should hand in a `zip` file containing your report (as a PDF, about 2-3 pages) along with the project containing your implementation. Make sure to clean (run `./gradlew clean`) any unnecessary garbage files before zipping. If your zip-file is very large, you likely did not clean.

7 Where to Submit

In IN2031, we use Devilry, not Canvas, for assignments. Go to

https://devilry.ifi.uio.no

and log in with your user UiO-name and password. When you are logged in, it should be fairly obvious how to upload the report and code.